Unit V

Data Visualizations

By

Team – Essentials of Data Science

School of Computer Engineering,
MIT Academy of Engineering, Alandi(D.)

Population and Samples

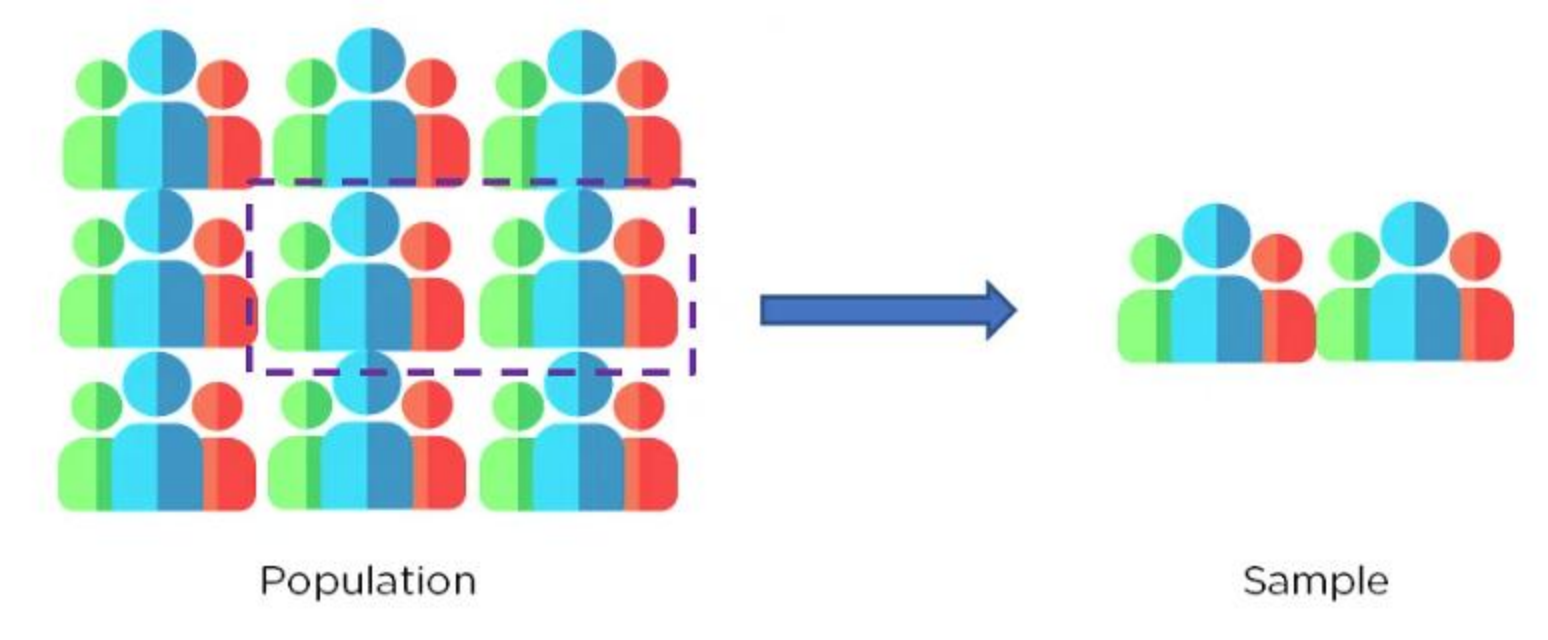
What is Population?

In statistics, population is the entire set of items from which you draw data for a statistical study. It can be a group of individuals, a set of items, etc. It makes up the data pool for a study.
Generally, population refers to the people who live in a particular area at a specific time. But in statistics, population refers to data on your study of interest. It can be a group of individuals, objects, events, organizations, etc. You use populations to draw conclusions.

What is a Sample?

A sample is defined as a smaller and more manageable representation of a larger group. A subset of a larger population that contains characteristics of that population. A sample is used in statistical testing when the population size is too large for all members or observations to be included in the test.
The sample is an unbiased subset of the population that best represents the whole data.

Population and Samples



Population and Samples

Samples are used when :

•The population is too large to collect data.

•The data collected is not reliable.

•The population is hypothetical and is unlimited in size. Take the example of a study that documents the results of a new medical procedure. It is unknown how the procedure will affect people across the globe, so a test group is used to find out how people react to it.

A sample should generally :

•Satisfy all different variations present in the population as well as a well-defined selection criterion.

•Be utterly unbiased on the properties of the objects being selected.

•Be random to choose the objects of study fairly.

Statistical Analysis of Data

- Statistical analysis is the process of collecting and analyzing data in order to discern patterns and trends.
- It is a method for removing bias from evaluating data by employing numerical analysis.
- This technique is useful for collecting the interpretations of research, developing statistical models, and planning surveys and studies.

Type of Statistical Analysis of Data

- Descriptive Analysis

Descriptive statistical analysis involves collecting, interpreting, analyzing, and summarizing data to present them in the form of charts, graphs, and tables. Rather than drawing conclusions, it simply makes the complex data easy to read and understand.

•Inferential Analysis

The inferential statistical analysis focuses on drawing meaningful conclusions on the basis of the data analyzed. It studies the relationship between different variables or makes predictions for the whole population.

•Predictive Analysis

Predictive statistical analysis is a type of statistical analysis that analyzes data to derive past trends and predict future events on the basis of them. It uses machine learning algorithms, data mining, data modelling, and artificial intelligence to conduct the statistical analysis of data.

Type of Statistical Analysis of Data

•Prescriptive Analysis

The prescriptive analysis conducts the analysis of data and prescribes the best course of action based on the results. It is a type of statistical analysis that helps you make an informed decision.

•Exploratory Data Analysis

Exploratory analysis is similar to inferential analysis, but the difference is that it involves exploring the unknown data associations. It analyzes the potential relationships within the data.

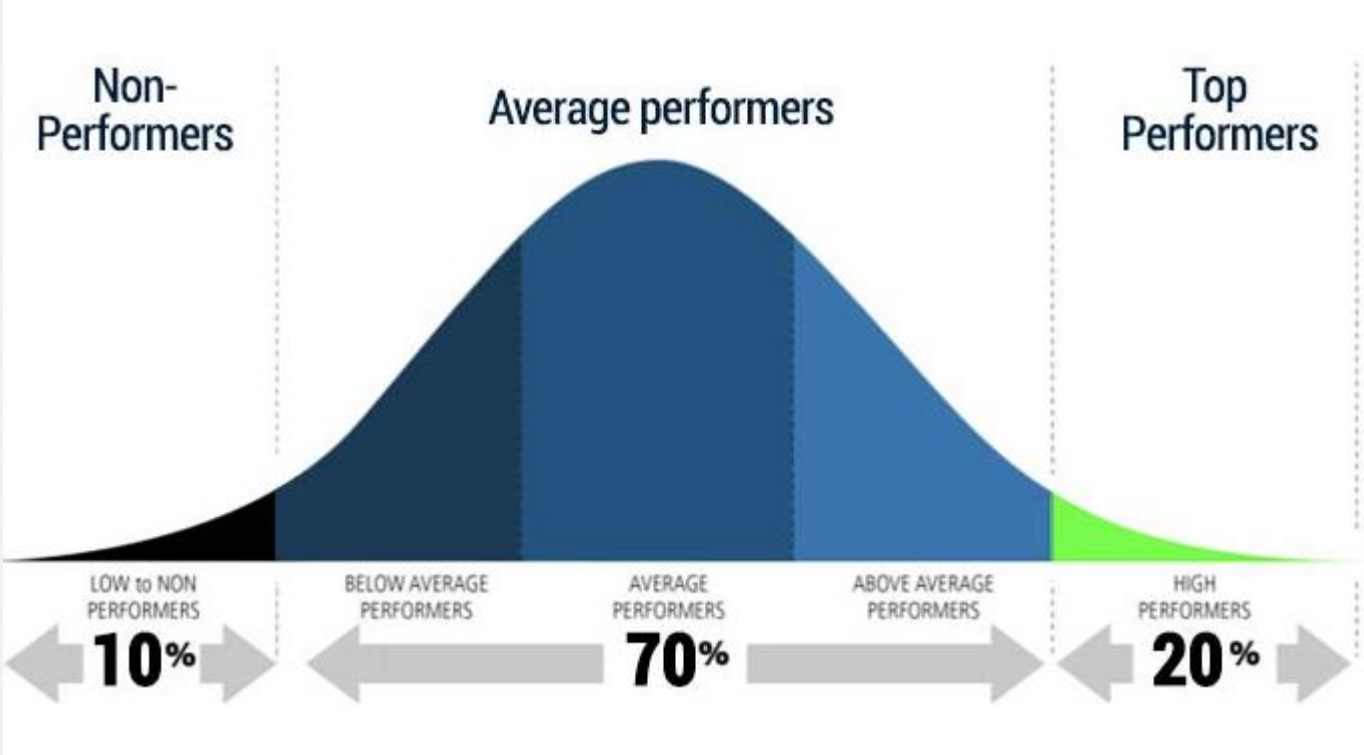
•Causal Analysis

The causal statistical analysis focuses on determining the cause and effect relationship between different variables within the raw data. In simple words, it determines why something happens and its effect on other variables. This methodology can be used by businesses to determine the reason for failure.

Importance of Statistical Analysis of Data

- The statistical analysis aids in summarizing enormous amounts of data into clearly digestible chunks.
- The statistical analysis aids in the effective design of laboratory, field, and survey investigations.
- Statistical analysis may help with solid and efficient planning in any subject of study.
- Statistical analysis aid in establishing broad generalizations and forecasting how much of something will occur under particular conditions.
- Statistical methods, which are effective tools for interpreting numerical data, are applied in practically every field of study. Statistical approaches have been created and are increasingly applied in physical and biological sciences, such as genetics.
- Statistical approaches are used in the job of a businessman, a manufacturer, and a researcher. Statistics departments can be found in banks, insurance businesses, and government agencies.
- A modern administrator, whether in the public or commercial sector, relies on statistical data to make correct decisions.
- Politicians can utilize statistics to support and validate their claims while also explaining the issues they address.

Normal Distribution Significance of Data Visualization in Data Science



Normal Distribution Significance of Data Visualization in Data Science

We call this **Bell-shaped curve** a **Normal Distribution.** Carl Friedrich Gauss discovered it so sometimes we also call it a **Gaussian Distribution** as well.

We can simplify the Normal Distribution's Probability Density by using only two parameters: **μ** Mean and **σ**2. This curve is **symmetric** around the Mean. Also as you can see for this distribution, the **Mean, Median, and Mode are all the same.**

One more important phenomena of a normal distribution is that it retains the normal shape throughout, unlike other probability distributions that change their properties after a transformation.

Importance of Normal Distribution

The normal distribution, also known as the Gaussian distribution or bell curve, plays a significant role in data science for several reasons. Here are some key reasons why the normal distribution is important in data science:

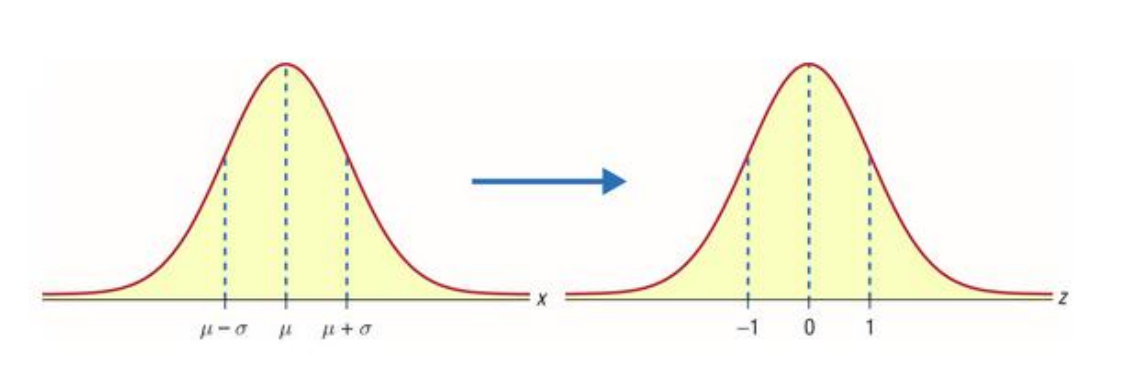
- Central Limit Theorem
- Modeling Data
- Statistical Inference
- Data Transformation
- Outlier Detection

Normal Distribution Significance of Data Visualization in Data Science

What is a Standard Normal Distribution?

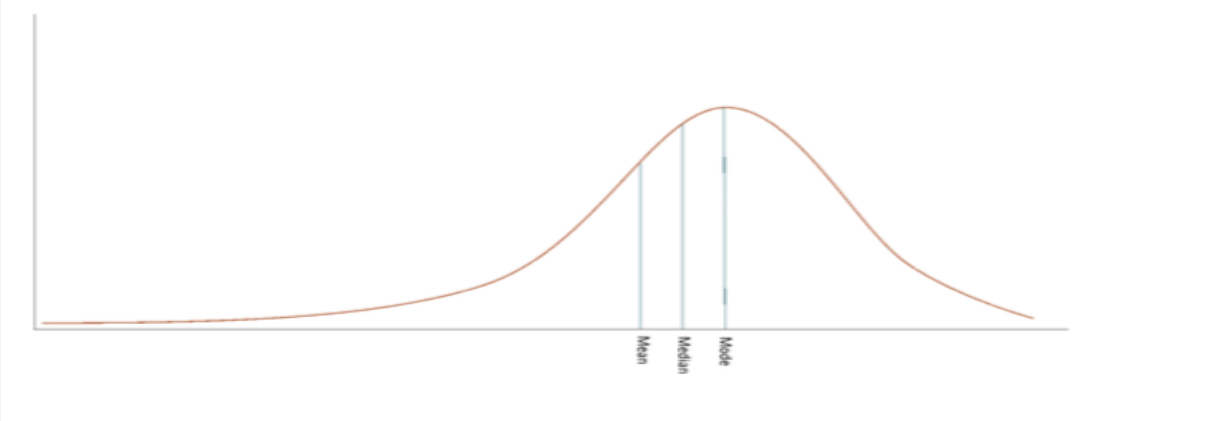
Standard Normal Distribution is a special case of Normal Distribution when $\mu = 0$ and $\sigma = 1$. For any Normal distribution, we can convert it into Standard Normal distribution using the formula:

$$Z = \frac{x - \mu}{\sigma}$$



Left Skewed Distribution

When data points cluster on the right side of the distribution, then the tail would be longer on the left side. This is the property of **Left Skewed Distribution**. The tail is longer in the negative direction so we also call it **Negatively Skewed Distribution.**

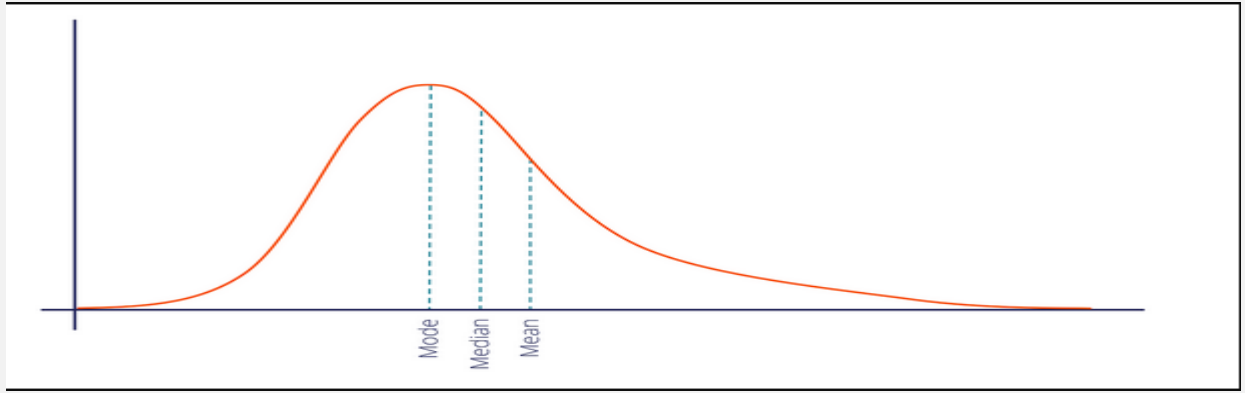


In the Normal Distribution, Mean, Median and Mode are equal but in a negatively skewed distribution, we express the general relationship between the central tendency measured as:

Mode > Median > Mean

Right Skewed Distribution

When data points cluster on the left side of the distribution, then the tail would be longer on the right side. This is the property of **Right Skewed Distribution**. Here, the tail is longer in the positive direction so we also call it **Positively Skewed Distribution.**



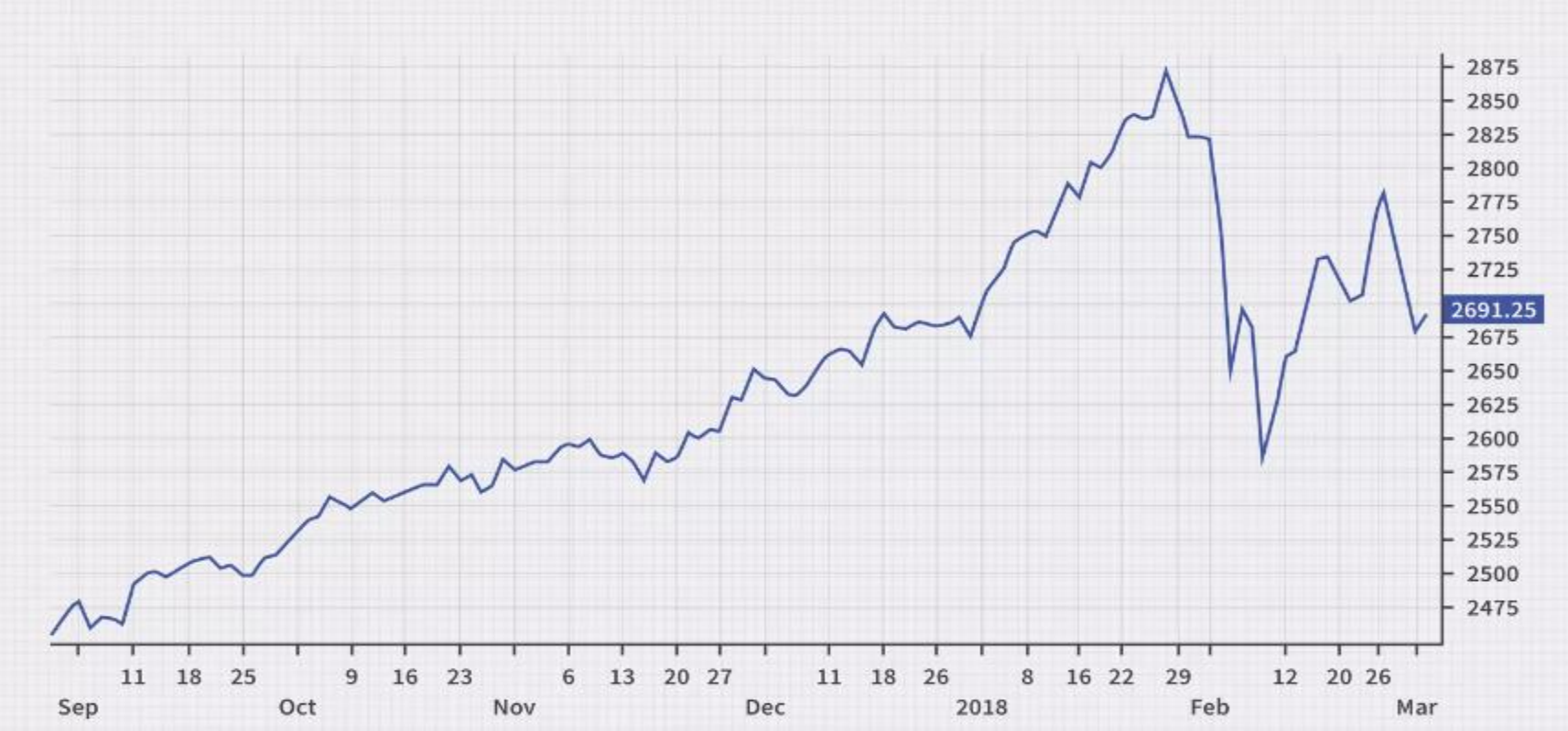
In a positively skewed distribution, we express the general relationship between the central tendency measures as:

Mode < Median < Mean

Line Chart

- A line chart is a graphical representation of an asset's historical price action that connects a series of data points with a continuous line.
- A line chart displays information as a series of data points connected by straight line segments.
- A line chart is a way of visually representing an asset's price history using a single, continuous line.
- Line charts usually only plot the closing prices, thus reducing noise from less critical times in the trading day, such as the open, high, and low prices.
- Line charts are simplistic and may not fully capture patterns or trends.

Line Chart

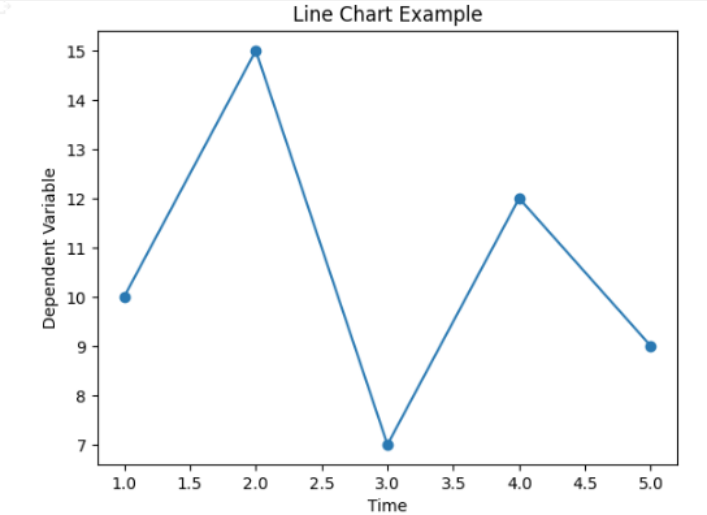

2875
2850
2825
2800
2775
2750
2725
2691.25
2675
2650
2625
2600
2575
2550
2525
2500
2475
11 18 25 9 16 23 6 13 20 27 11 18 26 8 16 22 29 12 20 26
Sep
Oct
Nov
Dec
2018
Feb
Mar

Plot Function

- **[x]** (optional): The x-coordinates of the data points. It can be a sequence or an array-like object. If **x** is not provided, the index of the **y** values w
- **y**: The y-coordinates of the data points. It can be a sequence or an array-like object.
- **[fmt]** (optional): The format string specifies the line style, marker style, and color of the plot. It is an optional parameter and can be used to cu
  - **'-'** or **'solid'**: Solid line
  - **'--'** or **'dashed'**: Dashed line
  - **':'** or **'dotted'**: Dotted line
  - **'-.'** or **'dashdot'**: Dash-dot line
  - **'.'**: Point markers
  - **'o'**: Circle markers
  - **'s'**: Square markers
  - **'r'**: Red color
  - **'g'**: Green color
  - **'b'**: Blue color
- **data** (optional): If specified, it overrides the default behavior of retrieving the data from the current axes. It can be a dictionary, Pandas DataFr
- ****kwargs** (optional): Additional keyword arguments can be provided to customize the plot. Some commonly used kwargs include:
  - **color**: Specifies the color of the line or markers. It can be a named color, a hex color code, or a tuple of RGB values.
  - **linewidth** (or **lw**): Specifies the line width in points.
  - **marker**: Specifies the marker style for scatter plots.
  - **markersize**: Specifies the size of markers in points.
  - **label**: Specifies the label for the plot, which can be used for creating a legend.
  - **alpha**: Specifies the transparency of the plot, ranging from 0 (completely transparent) to 1 (completely opaque).

Line Chart

```
import matplotlib.pyplot as plt

# Sample data
x = [1, 2, 3, 4, 5]  # x-axis values (time)
y = [10, 15, 7, 12, 9]  # y-axis values (dependent
variable)

# Create a line chart
plt.plot(x, y, marker='o')

# Customize the chart
plt.title("Line Chart Example")
plt.xlabel("Time")
plt.ylabel("Dependent Variable")

# Display the chart
plt.show()
```



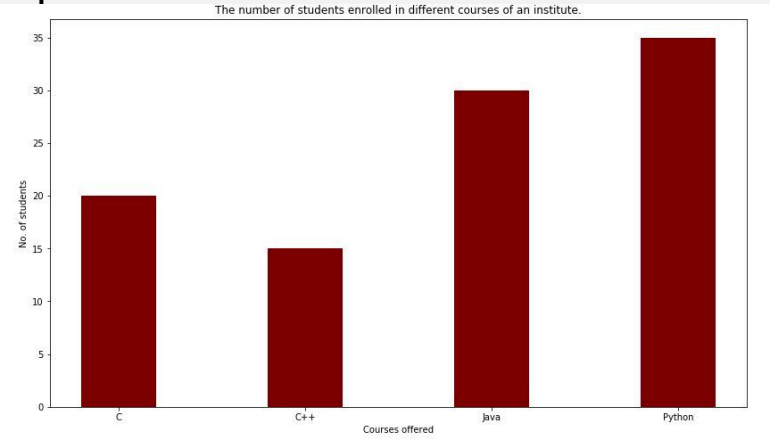
Line Chart – Pros and Cons

Pros
Trend Visualization
Relationship Between Variables
Simplicity and Clarity
Compact and Space Efficient

Cons
Limited Representation of Categorical Data
Lack of Precision for Small Datasets
Ignoring Data Distribution
Overemphasis on Long-Term Trends

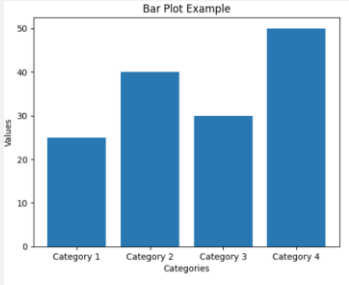
Bar Plot

- A bar plot or bar chart is a graph that represents the category of data with rectangular bars with lengths and heights that is proportional to the values which they represent. The bar plots can be plotted horizontally or vertically. A bar chart describes the comparisons between the discrete categories. One of the axis of the plot represents the specific categories being compared, while the other axis represents the measured values corresponding to those categories.

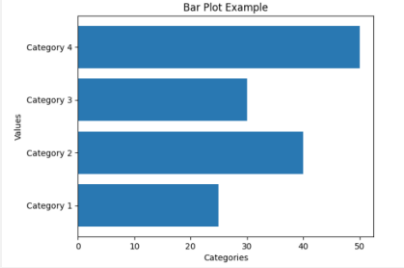


Bar Plot

```
import matplotlib.pyplot as plt

# Sample data
categories = ['Category 1', 'Category 2', 'Category 3',
'Category 4']
values = [25, 40, 30, 50]

# Create a bar plot
plt.bar(categories, values)

# Customize the plot
plt.title("Bar Plot Example")
plt.xlabel("Categories")
plt.ylabel("Values")

# Display the plot
plt.show()
```




```
import matplotlib.pyplot as plt

# Sample data
categories = ['Category 1', 'Category 2', 'Category 3'
'Category 4']
values = [25, 40, 30, 50]

# Create a bar plot
plt.barh(categories, values)

# Customize the plot
plt.title("Bar Plot Example")
plt.xlabel("Categories")
plt.ylabel("Values")

# Display the plot
plt.show()
```



Bar Plot

Advantages:

•Bar graph summarises the large set of data in simple visual form.
•It displays each category of data in the frequency distribution.
•It clarifies the trend of data better than the table.
•It helps in estimating the key values at a glance.

Disadvantages:

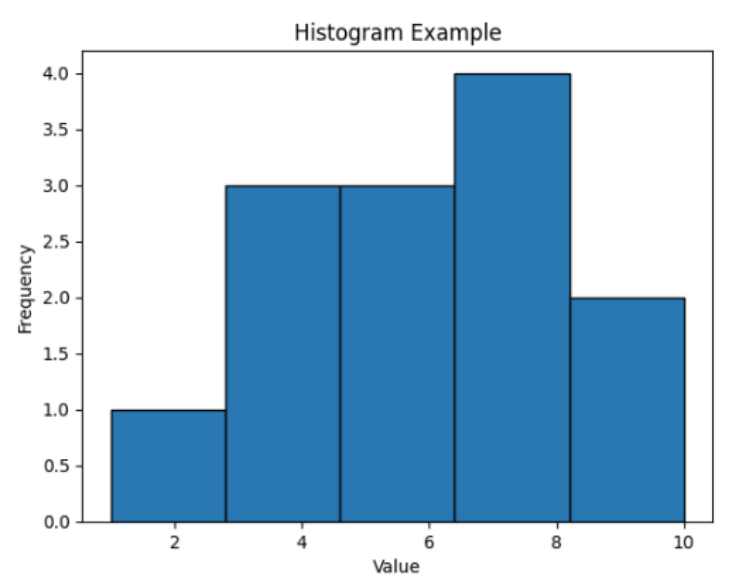
•Sometimes, the bar graph fails to reveal the patterns, cause, effects, etc.
•It can be easily manipulated to yield fake information.

Histogram

- A histogram is a graphical representation of the distribution of a dataset. It displays the frequencies or counts of data points within predefined intervals, known as bins, along the x-axis. The height of each bar in the histogram represents the frequency or count of data points falling within that particular bin.
- The primary purpose of a histogram is to visually illustrate the distribution and pattern of a dataset. It allows us to observe the range, central tendency, and variability of the data. By examining the shape of the histogram, we can identify patterns such as symmetry, skewness, or multimodality in the data.
- Histograms are commonly used for continuous or numerical data, although they can also be adapted for categorical or discrete data by grouping the categories into bins. They are particularly useful when dealing with large datasets or when exploring the overall distribution of a dataset.
- Histograms provide a visual summary of the data, making it easier to understand and interpret the underlying characteristics of the dataset. They are widely used in statistics, data analysis, and data visualization to gain insights, identify outliers, detect patterns, and make informed decisions based on the data's distribution.

Histogram

```
import matplotlib.pyplot as plt

# Example data
data = [1, 3, 4, 4, 5, 5, 6, 7, 8, 8, 8, 9, 10]

# Plotting the histogram
plt.hist(data, bins=5, edgecolor='black')

# Adding labels and title
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title('Histogram Example')

# Displaying the histogram
plt.show()
```



Histogram

Advantages of Histograms:
1. Visualizing Data Distribution
2. Easy Interpretation
3. Bin Flexibility
4. Useful for Continuous Data

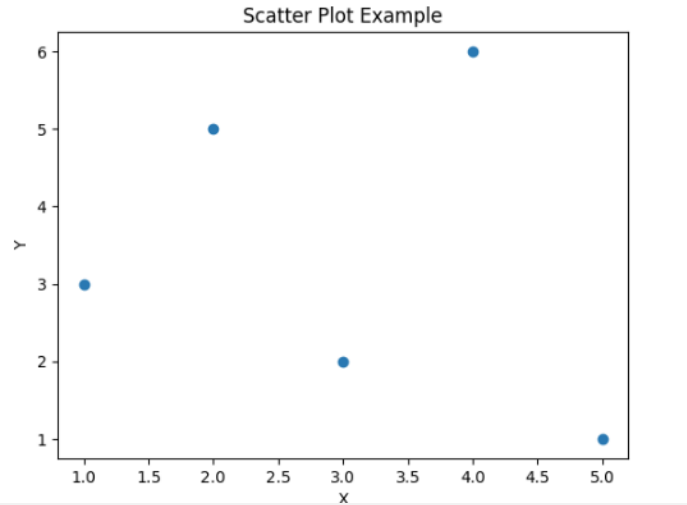
Disadvantage
1. Bin Selection Bias:
2. Information Loss
3. Sensitivity to Bin Width
4. Lack of Multivariate Analysis

Scatter Plot

- A scatter plot is a type of data visualization that displays the relationship between two variables. It consists of a set of points plotted on a two-dimensional plane, with each point representing a unique data observation. The position of each point on the plot corresponds to the values of the two variables being compared.
- In a scatter plot, one variable is plotted on the x-axis (horizontal axis) and the other variable is plotted on the y-axis (vertical axis). Each data point is then plotted at the intersection of its respective x and y values.
- The main purpose of a scatter plot is to visually examine the relationship or association between the two variables

Scatter Plot

```
import matplotlib.pyplot as plt

# Example data
x = [1, 2, 3, 4, 5]  # x-values
y = [3, 5, 2, 6, 1]  # y-values

# Plotting the scatter plot
plt.scatter(x, y)

# Adding labels and title
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Scatter Plot Example')

# Displaying the scatter plot
plt.show()
```



Scatter Plot

Advantages of Scatter Plots:
1. Relationship Visualization
2. Outlier Detection
3. Multivariate Analysis
4. Correlation Assessment

Disadvantages of Scatter Plots:
1. Limited to Continuous Variables
2. Overplotting
3. Complexity in Interpretation
4. Limited for Causal Inference

Pie Chart

A pie chart is a circular statistical graph that is divided into sectors, or "slices," representing the relative proportions or percentages of different categories within a dataset. Each slice of the pie chart corresponds to a specific category, and the size of each slice represents the proportionate value of that category relative to the whole.

Pie charts are commonly used to display categorical data, where the whole pie represents the total or 100% of the data. The length of each slice is determined by the proportion of the category it represents compared to the total.

The primary purpose of a pie chart is to provide a visual representation of the composition or distribution of different categories within a dataset. It allows for a quick and easy comparison of the relative sizes of each category, highlighting the most prominent or significant categories.

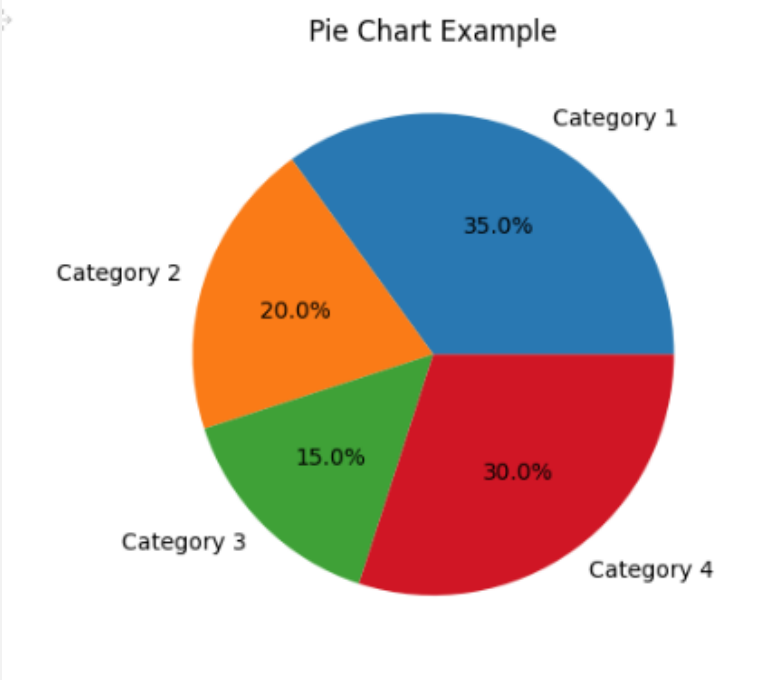
Pie charts are particularly useful when the number of categories is small (usually less than six or seven) and when the differences between categories are distinct. They are widely used in fields such as business, marketing, and data analysis to visually present market shares, survey responses, budget allocations, and other similar data.

It's worth noting that pie charts have some limitations. They can be less effective when the number of categories is large, as the slices can become too small and difficult to distinguish. Additionally, pie charts can be misleading if the proportions or angles of the slices are not accurately represented, making it challenging to interpret the exact values or make precise comparisons.

Overall, pie charts provide a straightforward and intuitive way to visualize the distribution of categorical data, allowing viewers to grasp the relative proportions of different categories at a glance.

Pie Chart

```
import matplotlib.pyplot as plt

# Example data
categories = ['Category 1', 'Category 2', 'Category 3',
'Category 4']
values = [35, 20, 15, 30]

# Plotting the pie chart
plt.pie(values, labels=categories, autopct='%1.1f%%')

# Adding a title
plt.title('Pie Chart Example')

# Displaying the pie chart
plt.show()
```



Pie Chart

Advantages of Pie Charts:

1. Visualizing Proportions
2. Simplicity and Intuitiveness
3. Highlighting Dominant Categories
4. Single Whole Representation

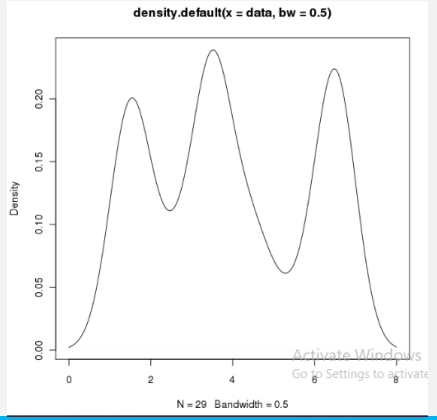
Disadvantages of Pie Charts:

1. Limited to Few Categories
2. Difficulty in Comparisons
3. Misleading Visual Perception

Density Plot

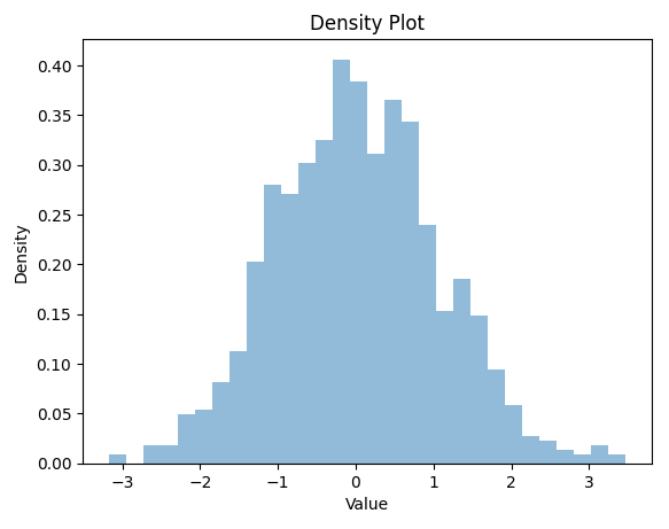
- Density Plot is a type of data visualization tool. It is a variation of the histogram that uses 'kernel smoothing' while plotting the values. It is a continuous and smooth version of a histogram inferred from a data.

- Density plots uses Kernel Density Estimation (so they are also known as Kernel density estimation plots or KDE) which is a probability density function. The region of plot with a higher peak is the region with maximum data points residing between those values.



Density Plot

```
import matplotlib.pyplot as plt
import numpy as np

# Generate some random data
data = np.random.randn(1000)

# Create a density plot
plt.hist(data, density=True, bins=30, alpha=0.5)

# Add labels and title
plt.xlabel('Value')
plt.ylabel('Density')
plt.title('Density Plot')

# Show the plot
plt.show()
```



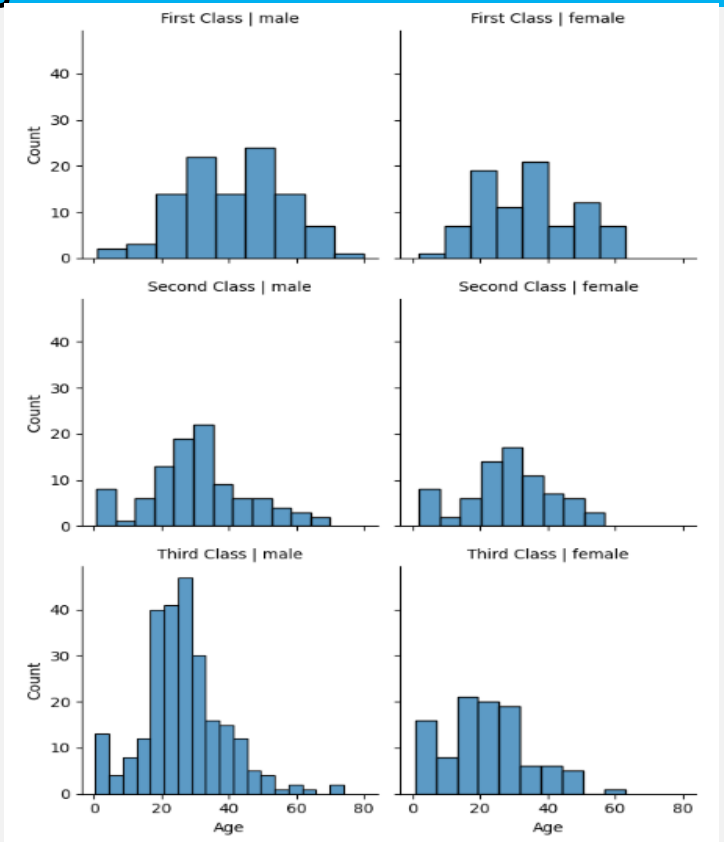
Facet Grids for categorical data

- A facet grid is a visualization technique that allows you to create a grid of subplots, where each subplot represents a subset of your data based on one or more categorical variables. It enables you to examine and compare different segments of your dataset across multiple dimensions.
- In a facet grid, the rows and columns represent different levels or categories of the variables you choose. Each cell in the grid corresponds to a unique combination of the variables, and you can plot different visualizations or charts within each cell to explore patterns and relationships.
- Facet grids are particularly useful when you want to examine the relationship between variables within different subsets of your data simultaneously. By organizing the data into a grid, you can easily compare and contrast the distributions, relationships, or trends across various categorical groups.

Facet Grids for categorical data

```
import seaborn as sns

# Load the example Titanic dataset
titanic = sns.load_dataset('titanic')

# Create a FacetGrid with two categorical variables: class and sex
grid = sns.FacetGrid(titanic, row='class', col='sex')

# Map a plot type to the grid using the desired variable: age
grid.map(sns.histplot, 'age')

# Set common labels for y-axis and x-axis
grid.set_axis_labels('Age', 'Count')

grid.set_titles(row_template='{row_name} Class', col_template='{col_name}')

plt.show() # Show the plot
```

Facet Grids for categorical data



Facet Grids for categorical data

Advantages of Facet Grid:

1. Enhanced Comparisons
2. Multidimensional Insights
3. Efficient Communication

Disadvantages of Facet Grid:

1. Limited Space
2. Potential Overcrowding
3. Complexity of Interpretation:

Group Plots

Group plots refer to the technique of organizing multiple plots or visualizations together to present information in a cohesive and meaningful manner. Grouping plots allows for easier comparison and analysis of different aspects of the data. There are several ways to achieve group plots, including:

1.Subplots: Subplots involve creating a grid of smaller plots within a larger plot area. With Matplotlib or other plotting libraries, you can use functions like **plt.subplots()** or **plt.subplot()** to create subplots and specify their arrangement (e.g., rows and columns). Each subplot can represent a different aspect of the data, allowing for side-by-side comparison.

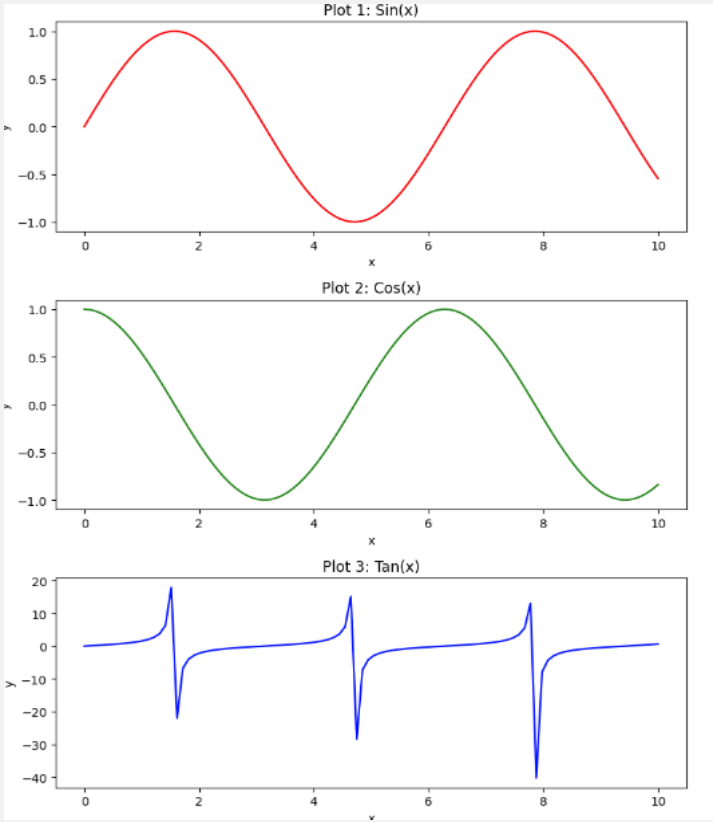
2.Facet Grids: Facet grids, commonly implemented in libraries like Seaborn, allow for organizing multiple subplots based on categorical variables. You can create a grid of subplots, where each subplot represents a unique combination of categorical variables. This facilitates comparisons between different categories and provides insights into relationships within the data.

3.GridSpec: Matplotlib's **GridSpec** module provides a powerful way to create complex subplot layouts. It allows you to define a grid with different-sized cells and position plots within those cells. This flexibility enables you to group plots in custom arrangements and sizes, accommodating complex visualization needs.

4.Panels or Tabs: In certain interactive plotting environments, such as Plotly or Bokeh, you can create panels or tabs to group related plots together. Each panel or tab can display a different set of plots, allowing users to switch between them and explore different aspects of the data.

Group Plots

```
import matplotlib.pyplot as plt
import numpy as np

# Generate some random data
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)
y3 = np.tan(x)

# Create a figure with subplots
fig, axes = plt.subplots(3, 1, figsize=(8, 10))

# Plot data on each subplot
axes[0].plot(x, y1, color='red')
axes[0].set_title('Plot 1: Sin(x)')
axes[0].set_xlabel('x')
axes[0].set_ylabel('y')
```



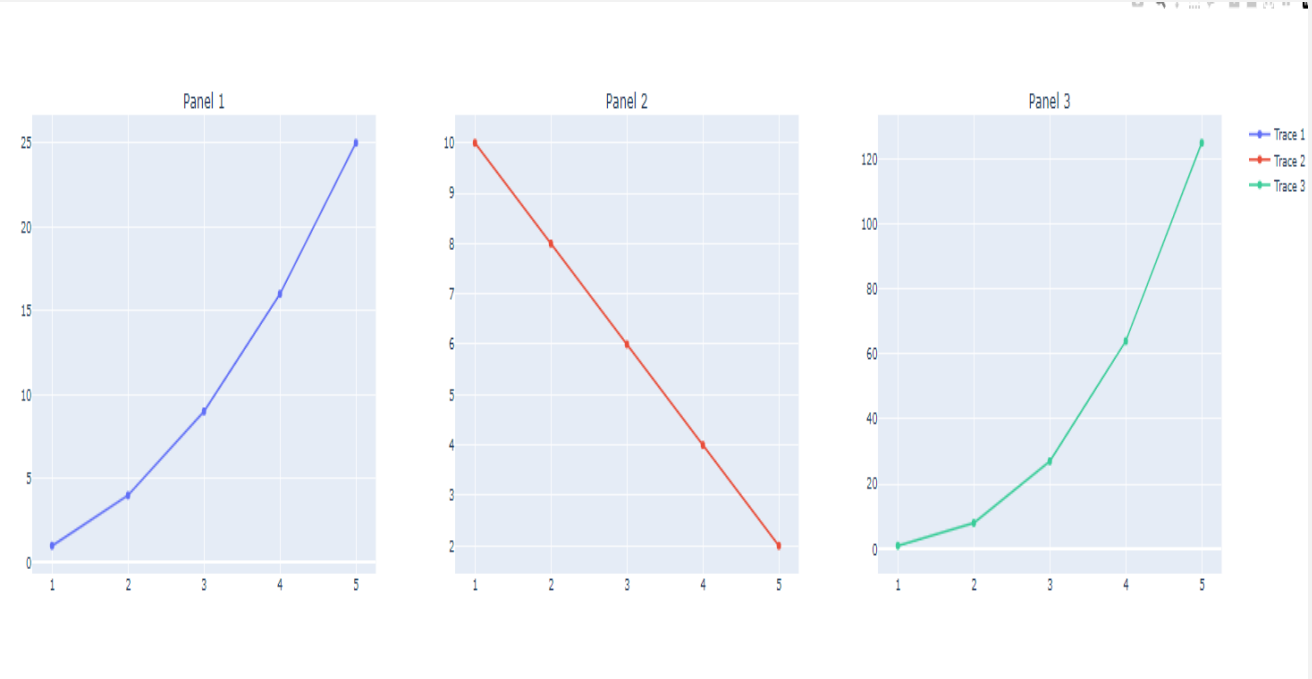
Group Plots

```
axes[1].plot(x, y2, color='green')
axes[1].set_title('Plot 2: Cos(x)')
axes[1].set_xlabel('x')
axes[1].set_ylabel('y')

axes[2].plot(x, y3, color='blue')
axes[2].set_title('Plot 3: Tan(x)')
axes[2].set_xlabel('x')
axes[2].set_ylabel('y')

# Adjust spacing between subplots
plt.tight_layout()

# Show the plot
plt.show()
```

Panels

```
import plotly.graph_objects as go
import plotly.subplots as sp

# Create data for the panels
x = [1, 2, 3, 4, 5]; y1 = [1, 4, 9, 16, 25]; y2 = [10, 8, 6, 4, 2]; y3 = [1, 8, 27, 64,
125]

# Create subplots with panels
fig = sp.make_subplots(rows=1, cols=3, subplot_titles=('Panel 1', 'Panel 2',
'Panel 3'))

# Add traces to each panel
fig.add_trace(go.Scatter(x=x, y=y1, name='Trace 1'), row=1, col=1)
fig.add_trace(go.Scatter(x=x, y=y2, name='Trace 2'), row=1, col=2)
fig.add_trace(go.Scatter(x=x, y=y3, name='Trace 3'), row=1, col=3)

# Update layout and display the plot
fig.update_layout(showlegend=True)
fig.show()
```

Panels

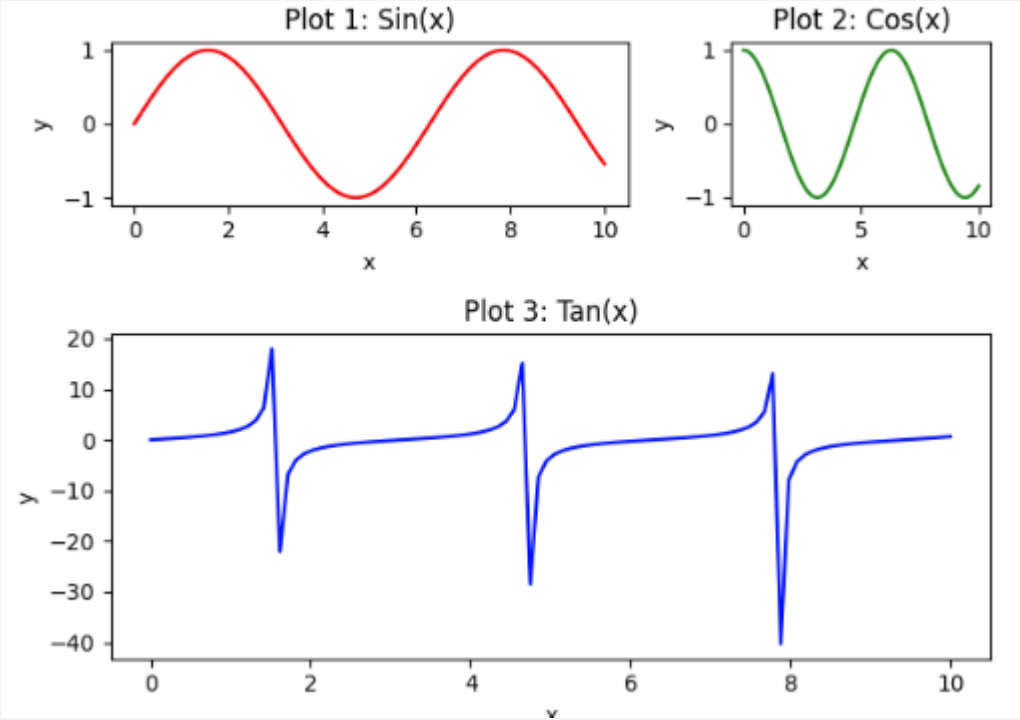


Panels

```
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import numpy as np

# Generate some random data
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)
y3 = np.tan(x)

# Create a GridSpec with different-sized cells
gs = gridspec.GridSpec(2, 2, width_ratios=[2, 1], height_ratios=[1, 2])

# Create subplots within the GridSpec
ax1 = plt.subplot(gs[0, 0])  # Top-left subplot
ax2 = plt.subplot(gs[0, 1])  # Top-right subplot
ax3 = plt.subplot(gs[1, :])  # Bottom row, spanning both columns
```

Panels

```
# Plot data on each subplot
ax1.plot(x, y1, color='red')
ax1.set_title('Plot 1: Sin(x)')
ax1.set_xlabel('x')
ax1.set_ylabel('y')

ax2.plot(x, y2, color='green')
ax2.set_title('Plot 2: Cos(x)')
ax2.set_xlabel('x')
ax2.set_ylabel('y')

ax3.plot(x, y3, color='blue')
ax3.set_title('Plot 3: Tan(x)')
ax3.set_xlabel('x')
ax3.set_ylabel('y')

plt.tight_layout() # Adjust spacing between subplots
plt.show() # Show the plot
```



Group Plots

Advantages of Group Plots:

1.Comparison
2.Storytelling
3.Contextualization

Disadvantages of Group Plots:

1.Complexity
2.Space Limitations
3. Interactivity Challenges
4. Plotting Consistency